

Modularization

rydnr

October 15, 2022

Copyright 2017 by rydnr.

The contents of this book are protected under the Creative Commons Attribution-ShareAlike 3.0 Unported license.

You are **free**:

- to **Share**: to copy, distribute and transmit the work,
- to **Remix**: to adapt the work,

Under the following conditions:

Attribution. You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike. If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

For any reuse or distribution, you must make clear to others the license terms of this work. The best way to do this is with a link to this web page:

<http://creativecommons.org/licenses/by-sa/3.0/>

Any of the above conditions can be waived if you get permission from the copyright holder. Nothing in this license impairs or restricts the author's moral rights.



Your fair dealing and other rights are in no way affected by the above. This is a human-readable summary of the Legal Code (the full license):

<http://creativecommons.org/licenses/by-sa/3.0/legalcode>

Contents

Illustrations	iii
----------------------	------------

I PharosEDA Modularization

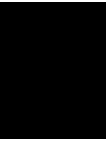
1 Introduction	3
1.1 Background	3
1.2 Objectives	3
2 Requirements	5
2.1 Inputs and outputs	5
2.2 Requirements format	5
2.3 Configuration	6
2.4 Checks	6
3 Configuration	7
3.1 First test	7
4 Reading specifications	11
4.1 Initial test	11
5 Chain of responsibility	17
5.1 Basic layout	17
5.2 Logging handler	18
5.3 Chain of Responsibility	19
6 Configuration handler	23
7 Spec handler	25
8 read class handler	29
8.1 A helper class to read Tonel files	29
8.2 A handler for reading a class	30
9 Write class handler	33
9.1 A helper class to write classes in Tonel format	33
9.2 A handler for writing a class	34

10	Composing handlers to read and write classes	37
10.1	A handler to iterate the specs	37
10.2	Composing handlers to read and write the classes	40
10.3	Minor fix	43
10.4	Fixing the specification format	45
11	Changing the package and category of the class	55
11.1	Updating the specification to include the target category	58
11.2	New handler for overwriting the category	68
12	Renaming references	77
13	Writing the specs	79
13.1	Missing classes	82
13.2	Classes not yet migrated	82
13.3	Extensions	82
	Bibliography	85

Illustrations

Part I

PharoEDA Modularization



Introduction

PharoEDA is already composed of different modules. The purpose of this refactoring is to split those modules into different repositories.

1.1 Background

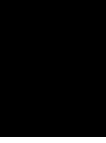
This refactoring consists of:

- Renaming classes
- Moving files across repositories.

However, PharoEDA (~develop~ branch) is evolving, and the renamed classes need to evolve as well.

1.2 Objectives

- Write a tool that keeps Modularized PharoEDA in sync with PharoEDA-develop.



Requirements

As described in the introduction, we're trying to implement a tool to allow as make Modularized PharoEDA in sync with PharoEDA-develop.

2.1 Inputs and outputs

Thanks PharoEDA is already in Tonel format, the inputs to the tool are

- package name
- class name
- class contents

The outputs to the tool are the following

- module (repository) name
- package name
- class name
- class contents

Obviously, the package name, class name and contents won't be the same.

2.2 Requirements format

One of the simplest formats we can define is just a table:

```
[ | input package | input class name | output module | output package
  | output class name |
  | | | | ]
```

We're omitting the class contents since they'll be copied verbatim, after translating class references.

Also, we're free to assume the input is a file in which every row is a comma-separated value with all of the expected columns.

2.3 Configuration

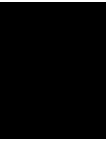
For simplicity, the tool won't know anything about git or version control systems. However, it'll need some configuration settings:

```
[ | module | folder | ]
```

As before, the input can be a file in which every line corresponds to a module, formatted as a CSV.

2.4 Checks

In case the tool, when trying to synchronize a class, finds that the contents of the class don't match, i.e., they have evolved independently, it should avoid changing the class, annotate the offending class, and report all problems at the end of the process.



Configuration

Let's start our tool by implementing the configuration layer.

As defined in Chapter 2, it needs to read a file such as

```
[ pharo-eda-adapters,/tmp/pharo-eda-adapters/src
```

Each row contains two fields: the name of a module, and a folder.

3.1 First test

The first is simple: given a file, interpret its contents.

We'll first create a package for the test: EDA-Multimodule-Migration-Test.

The first test will be defined in EDAModuleFileTest.

```
TestCase subclass: #EDAModuleFileTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'
```

We can now define the first test.

```
{ #category : #tests }
EDAMModuleFileTest >> testFileReadWithOneRow
| file entries |
file := FileSystem workingDirectory / 'tests' / 'modules1.csv'.
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'module1';
      nextPut: $,;
      nextPutAll: '/tmp/module1/src';
```

```

        nextPut: Character cr;
        nextPut: Character lf ].
entries := EDAMModuleFile new readFile: file.
self assert: entries isNotNil.
entries
do: [ :entry |
    self assert: (entry at: 'module') equals: 'module1'.
    self assert: (entry at: 'path') equals: '/tmp/module1/src' ]

```

We need to define the new class.

```

Object subclass: #EDAMModuleFile
instanceVariableNames: ''
classVariableNames: ''
package: 'EDA-Multimodule-Migration'

```

Running the test will force Pharo to suggest creating the missing method `readFile:` for us. Click on "Create" on the debugger, choose "EDAModule-File", and use "reading" as protocol. Afterwards, click on "Browse" and fill in the following code:

```

{ #category : #reading }
EDAMModuleFile >> readFile: aFileReference
^ OrderedCollection
with:
    (Dictionary new
     add: 'module' -> 'module1';
     add: 'path' -> '/tmp/module/src';
     yourself)

```

Of course, that's the naive implementation. We need to add a new test case.

```

{ #category : #tests }
EDAMModuleFileTest >> testFileReadWithTwoRows
| file entries |
file := FileSystem workingDirectory / 'tests' / 'modules2.csv'.
file asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'module1';
        nextPut: $,;
        nextPutAll: '/tmp/module1/src';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'module2';
        nextPut: $,;
        nextPutAll: '/temp/module2/src';
        nextPut: Character cr;
        nextPut: Character lf ].
entries := EDAModuleFile new readFile: file.
self assert: entries isNotNil.
self assert: (entries first at: 'module') equals: 'module1'.

```

3.1 First test

```
self assert: (entries first at: 'path') equals: '/tmp/module1/src'.
self assert: (entries second at: 'module') equals: 'module2'.
self assert: (entries second at: 'path') equals:
    '/temp/module2/src'
```

This new test fails. It's time to fix our implementation of `readFile:`.

```
{ #category : #reading }
EDAMModuleFile >> readFile: aFileReference
| result |
result := OrderedCollection new.
aFileReference
    readStreamDo: [ :stream |
        [ stream atEnd ]
            whileFalse: [ | row |
                row := ',' split: stream nextLine.
                result
                    add:
                        (Dictionary new
                            add: 'module' -> row first;
                            add: 'path' -> row second;
                            yourself) ] ].
    ^ result
```

The tests now work. However, the API we just defined is not particularly useful. It'd be better to build a dictionary of `module -> path` entries instead. Let's change the tests first.

```
{ #category : #tests }
EDAMModuleFileTest >> testFileReadWithOneRow
| file entries |
file := FileSystem workingDirectory / 'tests' / 'modules1.csv'.
file asFileReference
    writeStreamDo: [ :stream |
        stream
            nextPutAll: 'module1';
            nextPut: $,;
            nextPutAll: '/tmp/module1/src';
            nextPut: Character cr;
            nextPut: Character lf ].
entries := EDAModuleFile new readFile: file.
self assert: entries isNotNil.
self assert: (entries includesKey: 'module1').
self assert: (entries at: 'module1') equals: '/tmp/module1/src'
```

```
{ #category : #tests }
EDAMModuleFileTest >> testFileReadWithTwoRows
| file entries |
file := FileSystem workingDirectory / 'tests' / 'modules2.csv'.
file asFileReference
    writeStreamDo: [ :stream |
```

```

stream
  nextPutAll: 'module1';
  nextPut: $,;
  nextPutAll: '/tmp/module1/src';
  nextPut: Character cr;
  nextPut: Character lf;
  nextPutAll: 'module2';
  nextPut: $,;
  nextPutAll: '/temp/module2/src';
  nextPut: Character cr;
  nextPut: Character lf ].
entries := EDAModuleFile new readFile: file.
self assert: entries isNotNil.
self assert: (entries includesKey: 'module1').
self assert: (entries at: 'module1') equals: '/tmp/module1/src'.
self assert: (entries includesKey: 'module2').
self assert: (entries at: 'module2') equals: '/temp/module2/src'

```

The change in `readFile:` is straightforward.

```

{ #category : #reading }
EDAMModuleFile >> readFile: aFileReference
| result |
result := Dictionary new.
aFileReference
  readStreamDo: [ :stream |
    [ stream atEnd ]
      whileFalse: [ | row |
        row := ',' split: stream nextLine.
        result add: row first -> row second ] ].
^ result

```



Reading specifications

Now that we have the basic logic to read the configuration settings, we can build the logic needed to read the specifications, i.e., which class should be copied where.

As a quickly reminder, we need to parse files with the following format:

```
[input package,input class,output module,output package,output class
```

For example,

```
[ EDA-Events,EDAEventSourcingEvent,pharo-eda-api,EDAAPI-Events,EDAEventSourcingEvent
```

4.1 Initial test

We need an initial test. But first, the test class, in EDA-Multimodule-Migration-Tests.

```
[ TestCase subclass: #EDAMMSpecFileTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'
```

The initial test would be similar as what we wrote for EDAModuleTool:

```
{ #category : #tests }
EDAMMSpecFileTest >> testFileReadWithOneRow
| file entries |
file := FileSystem workingDirectory / 'tests' / 'spec1.csv'.
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'EDA-Events';
```

```

        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf ].
entries := EDAMMSpecFile new readFile: file.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
    assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
    assert: (entries first at: 'output package')
    equals: 'EDAAPI-Events'.
self assert: (entries first includesKey: 'output class').
self
    assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'

```

Just after saving it, Pharo will ask to create the new EDAMMSpecFile class for us.

```

Object subclass: #EDAMMSpecFile
  instanceVariableNames: ''
  classVariableNames: ''
  category: 'EDA-Multimodule-Migration'

```

We'll proceed as usual: running the test, and let Pharo write the initial method (under the "reading" protocol as before).

```

{ #category : #reading }
EDAMMSpecFile >> readFile: aFileReference
^ OrderedCollection
  with:
    (Dictionary new
      add: 'input package' -> 'EDA-Events';
      add: 'input class' -> 'EDAEventSourcingEvent';
      add: 'output module' -> 'pharo-eda-api';

```

4.1 Initial test

```
add: 'output package' -> 'EDAAPI-Events';
add: 'output class' -> 'EDAEventSourcingEvent';
yourself)
```

To get rid of the naive implementation, let's add another test.

```
{ #category : #tests }
EDAMMSpecFileTest >> testFileReadWithTwoRows
| file entries |
file := FileSystem workingDirectory / 'tests' / 'spec1.csv'.
file asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'EDA-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'EDA-Voyage-EventStore';
        nextPut: $,;
        nextPutAll: 'EDAMongoEventStore';
        nextPut: $,;
        nextPutAll: 'pharo-eda-adapters';
        nextPut: $,;
        nextPutAll: 'EDAA-EventStore-Voyage';
        nextPut: $,;
        nextPutAll: 'EDAMongoEventStore';
        nextPut: Character cr;
        nextPut: Character lf ].
entries := EDAMigrationToolFile new readFile: file.
self assert: entries isNotNil.
self assert: entries isNotEmpty.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
    assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
```

```

self
  assert: (entries first at: 'output package')
  equals: 'EDAAPI-Events'.
self assert: (entries first includesKey: 'output class').
self
  assert: (entries first at: 'output class')
  equals: 'EDAEventSourcingEvent'.
self assert: entries size > 1.
self assert: (entries second includesKey: 'input package').
self
  assert: (entries second at: 'input package')
  equals: 'EDA-Voyage-EventStore'.
self assert: (entries second includesKey: 'input class').
self
  assert: (entries second at: 'input class')
  equals: 'EDAMongoEventStore'.
self assert: (entries first includesKey: 'output module').
self
  assert: (entries second at: 'output module')
  equals: 'pharo-eda-adapters'.
self assert: (entries second includesKey: 'output package').
self
  assert: (entries second at: 'output package')
  equals: 'EDAA-EventStore-Voyage'.
self assert: (entries second includesKey: 'output class').
self
  assert: (entries second at: 'output class')
  equals: 'EDAAMongoEventStore'

```

Now that we have a failing test, we can replace our implementation so that it actually reads the contents from a file.

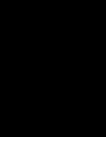
```

{ #category : #reading }
EDAMMSpecFile >> readFile: aFileReference
| result |
result := OrderedCollection new.
aFileReference
  readStreamDo: [ :stream |
    [ stream atEnd ]
    whileFalse: [ | row |
      row := ',' split: stream nextLine.
      result
        add:
          (Dictionary new
            add: 'input package' -> row first;
            add: 'input class' -> row second;
            add: 'output module' -> row third;
            add: 'output package' -> row fourth;
            add: 'output class' -> row fifth;
            yourself) ] ].

```

4.1 Initial test

$i_L^{\wedge}$ result



Chain of responsibility

Now that we can read the specifications and the configuration files, we can try to process each specification.

One basic thing to do is to copy the file to its final destination. We won't be dealing with changes in the source code just yet.

However, to accomodate all steps confortably, we'll apply the Chain of Responsibility pattern.

5.1 Basic layout

The pattern suggests to model two base concepts: `CommandSender` and `CommandHandler`, and specific handlers as subclasses of the latter.

In our case, we don't need anything special regarding the `CommandSender`. Actually, we want a Chain of Responsibility to follow the Open/Closed principle: to be open for extension, and closed for modification. In other words, we want to be able to add new steps if necessary, without having to modify the existing code or steps.

We need a test to start with.

```

[TestCase subclass: #EDAMMCommandHandlerTest
 instanceVariableNames: ''
 classVariableNames: ''
 package: 'EDA-Multimodule-Migration-Tests'

{ #category : #tests }
EDAMMCommandHandlerTest >> testDummyCommandHandler
| handler |
handler := EDAMMDummyCommandHandler new.
```

```

handler handleCommand: nil within: nil.
self assert: handler commandHandled

```

You'll need to create the `EDAMMDummyCommandHandler` class.

```

EDAMMCommandHandler subclass: #EDAMMDummyCommandHandler
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'

```

Running the test will ask you to implement the `handleCommand:within:` method. Do it in `EDAMMCommandHandler`.

```

{ #category : 'command handling' }
EDAMMCommandHandler >> handleCommand: aCommand within: aContext
  self shouldBeImplemented

```

Now, you'll need to override it in `EDAMMDummyCommandHandler` as well.

```

{ #category : 'command handling' }
EDAMMDummyCommandHandler >> handleCommand: aCommand within: aContext
  self commandHandled: true.
  ^ false

```

The missing piece to make the test pass is the `commandHandled` method. We'll use an instance variable for that.

```

EDAMMCommandHandler subclass: #EDAMMDummyCommandHandler
  instanceVariableNames: 'commandHandled'
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'

```

Don't forget to create the accessors.

```

{ #category : 'accessing' }
EDAMMLoggingCommandHandler >> commandHandled
  ^ commandHandled

{ #category : 'accessing' }
EDAMMLoggingCommandHandler >> commandHandled: aBoolean
  commandHandled := aBoolean

```

To initialize the variable to false, we'll use the `initialize` method.

```

{ #category : 'initialization' }
EDAMMLoggingCommandHandler >> initialize
  super initialize.
  self commandHandled: false

```

The test now should pass.

5.2 Logging handler

Let's add a new test.

```
[ { #category : 'tests' }
EDAMMCommandHandlerTest >> testLoggingCommandHandler
| handler |
handler := EDAMMLoggingCommandHandler
on: (WriteStream on: String new).
handler handleCommand: nil within: nil.
self assert: (handler stream contents includesSubstring: 'nil')
```

As usual, this test will help us creating the new class `EDAMMLoggingCommandHandler`:

```
[ EDAMMCommandHandler subclass: #EDAMMLoggingCommandHandler
instanceVariableNames: 'stream'
classVariableNames: ''
package: 'EDA-Multimodule-Migration'
```

Let's not forget to create the accessors.

```
[ { #category : 'tests' }
EDAMMLoggingCommandHandler >> stream
^ stream

[ { #category : 'tests' }
EDAMMLoggingCommandHandler >> stream: anObject
stream := anObject
```

Now it's time to create the factory method.

```
[ { #category : 'tests' }
EDAMMLoggingCommandHandler class >> on: aWriteStream
^ self new
stream: aWriteStream;
yourself
```

The only missing piece is to override `handleCommand:within:` from `EDAMMCommandHandler`.

```
[ { #category : 'tests' }
EDAMMLoggingCommandHandler >> handleCommand: aCommand within:
aContext
self stream nextPutAll: aCommand printString.
^ false
```

The test should be passing now.

5.3 Chain of Responsibility

To define which handlers we want to get involved in the chain, we'll use a Composite pattern.

The following test defines how the composite would be used.

```

[ TestCase subclass: #EDAMMCompositeCommandHandlerTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'

[ { #category : 'tests' }
  EDAMMCompositeCommandHandlerTest >> testCompositeCommandHandler
  | handler |
  handler := EDAMMCompositeCommandHandler
    with: EDAMMDummyCommandHandler new
    with: (EDAMMLoggingCommandHandler on: (WriteStream on: String
      new)).
  self assert: (handler handleCommand: nil within: nil)

```

Create the new class as descendant of EDAMMCommandHandler, under EDA-Multimodule-Migration.

```

[ EDAMMCommandHandler subclass: #EDAMMCompositeCommandHandler
  instanceVariableNames: 'handlers'
  classVariableNames: ''
  category: 'EDA-Multimodule-Migration'

```

The test will suggest us to create the missing factory method with:with:.

```

[ { #category : 'instance creation' }
  EDAMMCompositeCommandHandler class >> with: handler1 with: handler2
  ^ self new
    addHandler: handler1;
    addHandler: handler2;
    yourself

```

The accessors are almost trivial.

```

[ { #category : 'accessing' }
  EDAMMCompositeCommandHandler >> handlers
  ^ handlers
    ifNil: [ self handlers: OrderedCollection new.
      handlers ]

[ { #category : 'accessing' }
  EDAMMCompositeCommandHandler >> handlers: aCollection
  handlers := aCollection

```

The new method addHandler: is straightforward too.

```

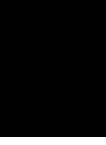
[ { #category : 'accessing' }
  EDAMMCompositeCommandHandler >> addHandler: aHandler
  self handlers add: aHandler

```

The only task we need to accomplish is overriding handleCommand:within:.

5.3 Chain of Responsibility

```
{ #category : 'command handling' }
EDAMMCompositeCommandHandler >> handleCommand: aCommand within:
    aContext
    ^ self handlers
        ifEmpty: [ false ]
        ifNotEmpty: [ :c | c anySatisfy: [ :h | h handleCommand:
            aCommand within: aContext ].
            true ]
```

Configuration handler

This handler would inject the configuration information into the context, when executing the chain of responsibility.

We'll reuse one of our previous tests to:

- create a file with the configuration;
- create a command containing the file reference;
- create a new `EDAMMConfigurationCommandHandler`;
- send the command to the handler;
- check the context contains the configuration settings from the file.

The body of the test is then:

```

]]]

%
    ${changes:path=code/002-EDAMMConfigurationCommandHandlerTest_testConfiguration
[[[classDefinition=false|lineNumber=true
EDAMMConfigurationCommandHandlerTest >>
    testConfigurationCommandHandler
| handler command context file entries |
file := FileSystem workingDirectory / 'tests' / 'modules1.csv'.
file asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'module1';
        nextPut: $,;
        nextPutAll: '/tmp/module1/src';
        nextPut: Character cr;
        nextPut: Character lf ].

```

```

context := Dictionary new.
command := Dictionary new.
command add: 'configuration file' -> file.
handler := EDAMMConfigurationCommandHandler new.
handler handleCommand: command within: context.
self assert: (context includesKey: 'configuration').
entries := context at: 'configuration'.
self assert: entries isNotNil.
self assert: (entries includesKey: 'module1').
self assert: (entries at: 'module1') equals: '/tmp/module1/src'

```

The new class can be defined as follows:

```

EDAMMCommandHandler subclass: #EDAMMConfigurationCommandHandler
instanceVariableNames: ''
classVariableNames: ''
package: 'EDA-Multimodule-Migration'

```

The test fails since we haven't overridden the `handleCommand:within:` method in our new handler. We'll skip the implementation of the naive version this time.

```

{ #category : 'command handling' }
EDAMMConfigurationCommandHandler >> handleCommand: aCommand within:
    aContext
| result |
result := false.
(self inputValid: aCommand)
    ifTrue: [ | fileReference |
        fileReference := aCommand at: 'configuration file'.
        aContext
            at: 'configuration'
            put: (EDAMMModuleFile new readFile: fileReference) ]
    ifFalse: [ result := true ].
^ result

```

We added a new checker method to ensure the required information is available.

```

{ #category : 'command checking' }
inputValid: aCommand
    ^ aCommand includesKey: 'configuration file'

```

The test passes. Any handler can now access the configuration settings from the context object, as long as the `EDAMMConfigurationCommandHandler` gets executed before.

Spec handler

This handler behaves in a similar way as `EDAMMConfigurationCommandHandler`. It's responsibility is to read the spec file and make it available in the context for other handlers to use.

```

TestCase subclass: #EDAMMSpecReaderCommandHandlerTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'

{ #category : 'tests' }
EDAMMSpecReaderCommandHandlerTest >> testSpecCommandHandler
| handler command context file entries |
file := FileSystem workingDirectory / 'tests' / 'spec1.csv'.
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'EDA-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: Character cr;
      nextPut: Character lf ].
context := Dictionary new.
command := Dictionary new
  add: 'specs file' -> file;
  yourself.

```

```

handler := EDAMMSpecCommandHandler new.
handler handleCommand: command within: context.
self assert: (context includesKey: 'specs').
entries := context at: 'specs'.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
    assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
    assert: (entries first at: 'output package')
    equals: 'EDA-API-Events'.
self assert: (entries first includesKey: 'output class').
self
    assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'

```

Pharo will ask us to create the new class right when we try to save the test itself.

```

EDAMMCommandHandler subclass: #EDAMMSpecCommandHandler
    instanceVariableNames: ''
    classVariableNames: ''
    category: 'EDA-Multimodule-Migration'

```

The `handleCommand:within:` implementation is very similar to `EDAMMConfigurationCommandHandler's`:

```

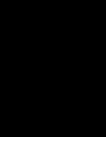
{ #category : 'command handling' }
EDAMMSpecCommandHandler >> handleCommand: aCommand within: aContext
| result |
result := false.
(self inputValid: aCommand)
    ifTrue: [ | fileReference |
        fileReference := aCommand at: 'specs file'.
        aContext
            at: 'specs'
            put: (EDAMMSpecFile new readFile: fileReference) ]
    ifFalse: [ result := true ].
^ result

```

As well as the `inputValid:` implementation.

```
[ { #category : 'command checking' }  
  EDAMMSpecCommandHandler >> inputValid: aCommand  
    ^ aCommand includesKey: 'specs file'
```

The test should pass.



read class handler

Now it's time to deal with the main purpose of this migration tool: copying classes to another directory. It'll perform a "file-out" on the source class, in Tonel format, to the output directory.

However, later we know we'll need to check if the class already exists in the destination, so it's better to start with the logic in charge of reading files in Tonel format.

8.1 A helper class to read Tonel files

We can define a simple test for reading a known class.

```
TestCase subclass: #EDAMMClassFileTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'
```

The test itself uses TonelReader to do the heavy lifting.

```
EDAMMClassFileTest >> testReadEDACCommandConsumer
| folder entry |
folder := FileSystem workingDirectory / 'src'.
entry := EDAMMClassFile new
  readClass: 'EDACCommandConsumer'
  inPackage: 'EDA-Messaging'
  inFolder: folder.
self assert: entry isNotNil
```

We need to create the new class.

```
Object subclass: #EDAMMClassFile
  instanceVariableNames: ''
  classVariableNames: ''
  category: 'EDA-Multimodule-Migration'
```

The implementation of `readClass:inPackage:inFolder:` is basically calling `ToneIReader`.

```
{ #category : 'reading' }
EDAMMClassFile >> readClass: aClassName inPackage: aPackageName
  inFolder: aFolder
  ^ (ToneIReader on: aFolder asFileReference fileName: aPackageName)
  definitions
  select: [ :def | def className = aClassName ]
```

Notice `ToneIReader` returns a list of definitions: one for the class itself, and one for each method.

8.2 A handler for reading a class

Using the helper class we just implemented, we can now write a handler that reads the class and binds it to the context for other commands to process.

This time we'll use a different test class.

```
TestCase subclass: #EDAMMClassFileReaderCommandHandlerTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'
```

The test is as follows:

```
{ #category : 'tests' }
EDAMMClassFileReaderCommandHandlerTest >>
  testClassFileReaderCommandHandler
  | handler command context definitions |
  command := Dictionary new
    add: 'class' -> 'EDACommandConsumer';
    add: 'package' -> 'EDA-Messaging';
    add: 'source root' -> (FileSystem workingDirectory / 'src');
    yourself.
  context := Dictionary new.
  handler := EDAMMClassFileReaderCommandHandler new.
  handler handleCommand: command within: context.
  self assert: (context includesKey: 'EDACommandConsumer').
  definitions := (context at: 'EDACommandConsumer').
  self assert: definitions isNotNil.
  self assert: definitions isNotEmpty.
  definitions do: [ :def | self assert: def className equals:
    'EDACommandConsumer' ]
```

The new class `EDAMMClassFileReaderCommandHandler` is a simple stateless class.

```
EDAMMCommandHandler subclass: #EDAMMClassFileReaderCommandHandler
    instanceVariableNames: ''
    classVariableNames: ''
    category: 'EDA-Multimodule-Migration'
```

Its `handleCommand:within:` makes sure it stops the processing altogether if the class is not found.

```
{ #category : 'command handling' }
EDAMMClassFileReaderCommandHandler >> handleCommand: aCommand
    within: aContext
    | result |
    result := false.
    (self inputValid: aCommand)
        ifTrue: [ | sourceRoot packageName className entries |
            sourceRoot := aCommand at: 'source root'.
            packageName := aCommand at: 'package'.
            className := aCommand at: 'class'.
            entries := EDAMMClassFile new
                readClass: className
                inPackage: packageName
                inFolder: sourceRoot.
            entries
                ifEmpty: [ result := true ]
                ifNotEmpty: [ aContext at: className put: entries ] ].
    ^ result
```

The checker method just ensures the command includes the expected information.

```
{ #category : 'command checking' }
EDAMMClassFileReaderCommandHandlerTest >> inputValid: aCommand
    ^ (aCommand includesKey: 'source root')
    and: [ (aCommand includesKey: 'package')
        and: [ aCommand includesKey: 'class' ] ]
```


Write class handler

We'll now proceed to do the same for writing classes to disk.

9.1 A helper class to write classes in Tonel format

As we did before, let's start with a new test, this time in the same EDAMM-ClassFileTest.

```
{ #category : 'tests' }
EDAMMClassFileTest >> testWriteEDACommandConsumer
| folder file defs |
folder := FileSystem workingDirectory / 'tests'.
defs := EDAMMClassFile new
    readClassNamed: 'EDACommandConsumer'
    inPackage: 'EDA-Messaging'
    inFolder: FileSystem workingDirectory / 'src'.
file := EDAMMClassFile new
    writeClassNamed: 'EDACommandConsumer'
    withDefinitions: defs
    inFolder: folder.
self assert: file isNotNil
```

The method just uses TonelWriter to perform the "file-out".

```
{ #category : 'writing' }
EDAMMClassFile >> writeClassNamed: aString withDefinitions:
    aCollection inFolder: aFileReference
aFileReference ensureCreateDirectory.
^ (aFileReference / aString , 'st')
    writeStreamDo: [ :stream |
        | writer classDefinition |
        writer := TonelWriter new.
```

```

classDefinition := (aCollection
  select: [ :def | def isClassDefinition ]) first.
writer writeClassDefinition: classDefinition on: stream.
(aCollection
  select: [ :def | def isClassDefinition not and: [ def
classIsMeta ] ])
  collect:
    [ :def | writer writeMethodDefinition: def parent:
classDefinition on: stream ].
(aCollection
  select: [ :def | def isClassDefinition not and: [ def
classIsMeta not ] ])
  collect:
    [ :def | writer writeMethodDefinition: def parent:
classDefinition on: stream ] ];
yourself

```

9.2 A handler for writing a class

Now we're ready to focus now on the handler to serialize a class in Tonel format.

```

TestCase subclass: #EDAMMClassFileWriterCommandHandlerTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'

{ #category : 'tests' }
EDAMMClassFileWriterCommandHandlerTest >>
  testClassFileReaderCommandHandler
| handler command context targetFolder outputFile |
targetFolder := FileSystem workingDirectory / 'tests'.
outputFile := targetFolder / 'EDACommandConsumer.st'.
outputFile exists
  ifTrue: [ outputFile delete ].
command := Dictionary new
  add: 'class' -> 'EDACommandConsumer';
  add:
    'EDACommandConsumer'
    ->
      (EDAMMClassFile new
        readClassNamed: 'EDACommandConsumer'
        inPackage: 'EDA-Messaging'
        inFolder: FileSystem workingDirectory / 'src');
  add: 'target folder' -> targetFolder;
  yourself.
context := Dictionary new.
handler := EDAMMClassFileWriterCommandHandler new.
handler handleCommand: command within: context.

```

9.2 A handler for writing a class

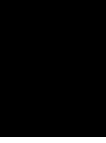
```
i self assert: outputFile exists
L
```

The test needs the following class to be created.

```
[ EDAMMCommandHandler subclass: #EDAMMClassFileWriterCommandHandler
  instanceVariableNames: ''
  classVariableNames: ''
  category: 'EDA-Multimodule-Migration'
```

The implementation of `handleCommand:within:` is similar to `EDAMMClassFileReaderCommandHandler`'s.

```
[ EDAMMClassFileWriterCommandHandler >> handleCommand: aCommand
  within: aContext
  | result |
  result := false.
  (self inputValid: aCommand)
  ifTrue: [ | className |
    className := aCommand at: 'class'.
    EDAMMClassFile new
      writeClassNamed: className
      withDefinitions: (aCommand at: className)
      inFolder: (aCommand at: 'target folder') ].
  ^ result
```

Composing handlers to read and write classes

With the components we have right now, we can compose them to build an initial solution to copy classes to their destination folders.

The idea is:

- Setup a chain with `EDAMMConfigurationCommandHandler` and `EDAMMSpecCommandHandler`.
- For each spec entry, build another chain with `EDAMMClassFileReaderCommandHandler` and `EDAMMClassFileWriterCommandHandler`.

We'll leave for later how to deal with changes in the name of the packages, and checking if the output can be overwritten.

10.1 A handler to iterate the specs

We'll create a new test for this handler.

```
TestCase subclass: #EDAMMSpecIteratorCommandHandlerTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'
```

The first test is simple: after reading the specs, we want the new handler to perform a block for each spec.

```

{ #category : 'tests' }
EDAMMSpecIteratorCommandHandlerTest >> testSpecIteratorCommandHandler
| handler command file specsFound |
file := FileSystem workingDirectory / 'tests' / 'spec1.csv'.
file asFileReference
    writeStreamDo: [ :stream |
        stream
            nextPutAll: 'EDA-Events';
            nextPut: $,;
            nextPutAll: 'EDAEventSourcingEvent';
            nextPut: $,;
            nextPutAll: 'pharo-eda-api';
            nextPut: $,;
            nextPutAll: 'EDAAPI-Events';
            nextPut: $,;
            nextPutAll: 'EDAEventSourcingEvent';
            nextPut: Character cr;
            nextPut: Character lf;
            nextPutAll: 'EDA-Domain';
            nextPut: $,;
            nextPutAll: 'EDAAggregateRoot';
            nextPut: $,;
            nextPutAll: 'pharo-eda-api';
            nextPut: $,;
            nextPutAll: 'EDAAPI-Aggregates';
            nextPut: $,;
            nextPutAll: 'EDAAggregateRoot';
            nextPut: Character cr;
            nextPut: Character lf ].
command := Dictionary new
    add: 'specs file' -> file;
    yourself.
specsFound := OrderedCollection new.
handler := EDAMMSpecIteratorCommandHandler
    for: [ :spec | specsFound add: spec ].
handler handleCommand: command within: Dictionary new.
self assert: specsFound isEmpty.
self assert: specsFound size equals: 2.
self
    assert: specsFound first
    equals:
        (Dictionary new
            add: 'input package' -> 'EDA-Events';
            add: 'input class' -> 'EDAEventSourcingEvent';
            add: 'output module' -> 'pharo-eda-api';
            add: 'output package' -> 'EDAAPI-Events';
            add: 'output class' -> 'EDAEventSourcingEvent';
            yourself).
self
    assert: specsFound second

```

10.1 A handler to iterate the specs

```
equals:
  (Dictionary new
    add: 'input package' -> 'EDA-Domain';
    add: 'input class' -> 'EDAAggregateRoot';
    add: 'output module' -> 'pharo-eda-api';
    add: 'output package' -> 'EDAAPI-Aggregates';
    add: 'output class' -> 'EDAAggregateRoot';
    yourself)
```

As always, Pharo asks us if we want it to create the new class `EDAMMSpecIteratorCommandHandler` for us.

```
EDAMMCommandHandler subclass: #EDAMMSpecIteratorCommandHandler
  instanceVariableNames: 'block'
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration'
```

We also want to generate the accessors for the block instance variable.

```
{ #category : 'accessing' }
block
  ^ block

{ #category : 'accessing' }
block: aBlock
  block := aBlock
```

We anticipated we'd like to create `EDAMMSpecIteratorCommandHandler` instances using a `for: factory` method.

Let's create it.

```
{ #category : 'instance creation' }
for: aBlockClosure
  ^ self new
    block: aBlockClosure;
    yourself
```

The only missing piece is implementing `handleCommand:within:`.

```
{ #category : 'command handling' }
EDAMMSpecIteratorCommandHandler >> handleCommand: aCommand within:
  aContext
  | specs |
  EDAMMSpecReaderCommandHandler new
    handleCommand: aCommand
    within: aContext.
  specs := newContext at: 'specs' ifAbsent: OrderedCollection new.
  specs do: [ :spec | self block value: spec value: aCommand value:
    aContext ].
  ^ false
```

The test now should pass.

10.2 Composing handlers to read and write the classes

- Setup a chain with `EDAMMConfigurationCommandHandler` and `EDAMMSpecCommandHandler`.
- For each spec entry, build another chain with `EDAMMClassFileReaderCommandHandler` and `EDAMMClassFileWriterCommandHandler`.

It's time to give shape to the tool itself. We'll name our new test suite after it.

```
TestCase subclass: #EDAMMMigrationToolTest
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration-Tests'
```

The test takes care of the following:

- creating a new temporary folder where `MigrationTool` will copy the classes,
- creating a configuration file to associate the temporary folder with the destination module,
- creating a specs file,
- running `MigrationTool`,
- checking if the temporary folder contains the copied classes.

We'll create a separate method to create a new empty folder for the test.

```
{ #category : 'helpers' }
EDAMMMigrationToolTest >> deleteFolder: aFolder
aFolder exists
  ifFalse: [ ^ self ].
aFolder asFileReference entries
  do: [ :f |
    f isDirectory
      ifTrue: [ self deleteFolder: f ]
      ifFalse: [ f reference delete ] ].
aFolder asFileReference delete
```

Now, the test itself.

```
{ #category : 'tests' }
EDAMMMigrationToolTest >> testMigrationTool
| outputFolder confFile specFile |
outputFolder := FileSystem root / 'tmp' / 'pharo-eda-api' / 'src'.
self deleteFolder: outputFolder.
outputFolder ensureCreateDirectory.
confFile := FileSystem workingDirectory / 'tests'
  / 'testMigrationTool.config.csv'.
confFile asFileReference
  writeStreamDo: [ :stream |
```

```

        stream
            nextPutAll: 'pharo-eda-api';
            nextPut: $,;
            nextPutAll: outputFolder path pathString;
            nextPut: Character cr;
            nextPut: Character lf ].
specFile := FileSystem workingDirectory / 'tests'
/ 'testMigrationTool.spec.csv'.
specFile asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'EDA-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'EDA-Domain';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Aggregates';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: Character cr;
        nextPut: Character lf ].
EDAMMMigrationTool new
    processSpecsFile: specFile
    withConfigurationFile: confFile.
self assert: (outputFolder / 'EDAAPI-Events') exists.
self
    assert: (outputFolder / 'EDAAPI-Events' /
        'EDAEventSourcingEvent.st') exists.
self assert: (outputFolder / 'EDAAPI-Aggregates') exists.
self
    assert: (outputFolder / 'EDAAPI-Aggregates' /
        'EDAAggregateRoot.st') exists

```

As always, we need to create the class just when saving the method.

```
Object subclass: #EDAMMMigrationTool
  instanceVariableNames: ''
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration'
```

If you run the test now, Pharo will ask you to first to create the method `processSpecsFile:withConfigurationFile:.`

```
{ #category : 'api' }
processSpecsFile: aSpecsFileReference withConfigurationFile:
  aConfFileReference
  | handler command |
  command := Dictionary new
    add: 'specs file' -> aSpecsFileReference;
    add: 'configuration file' -> aConfFileReference;
    yourself.
  handler := EDAMMCompositeCommandHandler
    with: EDAMMConfigurationCommandHandler new
    with:
      (EDAMMSpecIteratorCommandHandler
        for: [ :spec :cmd :ctx |
          self
            copySpec: spec
            to: ((ctx at: 'configuration') at: (spec at: 'output
              module')) ] ).
  handler handleCommand: command within: Dictionary new
```

Then, the test will require us to implement `copySpec:to:.` That method is just the glue needed to compose the required handlers together.

```
{ #category : 'copying' }
copySpec: aSpecDictionary to: aFolderPathAsString
  | result command context |
  command := Dictionary new
    add: 'class' -> (aSpecDictionary at: 'input class');
    add: 'package' -> (aSpecDictionary at: 'input package');
    add: 'source root' -> (FileSystem workingDirectory / 'src');
    yourself.
  context := Dictionary new.
  result := (EDAMMClassFileReaderCommandHandler new
    handleCommand: command
    within: context)
    ifTrue: [ true ]
    ifFalse: [ EDAMMClassFileWriterCommandHandler new
      handleCommand:
        (Dictionary new
          add: 'class' -> (aSpecDictionary at: 'input class');
          add:
            'target folder'
            ->
              (aFolderPathAsString asFileReference
```

```

        / (aSpecDictionary at: 'output package'));
        addAll: context;
        yourself)
        within: context ].
    ^ result

```

Finally, the test passes. We now have a tool able to copy files to the separate PharoEDA modules.

10.3 Minor fix

The tool is not necessarily coupled with PharoEDA, but for one detail: in `EDAMMMigrationTool >> copySpec:to:` we use `(FileSystem workingDirectory) / 'src'` as `**`. That means we cannot copy classes not part of PharoEDA. Also, it means that if we move `EDAMMMigrationTool` to a different repository, it won't be able to read the classes to copy.

One solution is pretty straightforward: we can just add `**` as another module in the configuration file.

The test would become:

```

{ #category : 'tests' }
EDAMMMigrationToolTest >> testMigrationTool
| outputFolder configFile specFile |
outputFolder := FileSystem root / 'tmp' / 'pharo-eda-api' / 'src'.
self deleteFolder: outputFolder.
outputFolder ensureCreateDirectory.
configFile := FileSystem workingDirectory / 'tests'
/ 'testMigrationTool.config.csv'.
configFile asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: (FileSystem workingDirectory / 'src') path
        pathString;
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: outputFolder path pathString;
        nextPut: Character cr;
        nextPut: Character lf ].
specFile := FileSystem workingDirectory / 'tests'
/ 'testMigrationTool.spec.csv'.
specFile asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'EDA-Events';

```

```

        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'EDA-Domain';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Aggregates';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: Character cr;
        nextPut: Character lf ].
EDAMMMigrationTool new
    processSpecsFile: specFile
    withConfigurationFile: confFile.
self assert: (outputFolder / 'EDAAPI-Events') exists.
self
    assert: (outputFolder / 'EDAAPI-Events' /
        'EDAEventSourcingEvent.st') exists.
self assert: (outputFolder / 'EDAAPI-Aggregates') exists.
self
    assert: (outputFolder / 'EDAAPI-Aggregates' /
        'EDAAggregateRoot.st') exists

```

`processSpecsFile:withConfigurationFile:` in `EDAMMMigrationTool` would use the input folder from the modules (configuration) file.

```

{ #category : 'api' }
EDAMMMigrationTool >> processSpecsFile: aSpecsFileReference
    withConfigurationFile: aConfFileReference
| handler command |
command := Dictionary new
    add: 'specs file' -> aSpecsFileReference;
    add: 'configuration file' -> aConfFileReference;
    yourself.
handler := EDAMMCompositeCommandHandler
    with: EDAMMConfigurationCommandHandler new
    with:
        (EDAMMSpecIteratorCommandHandler
            for: [ :spec :cmd :ctx |
                self

```

```

        copySpec: spec
        from: ((ctx at: 'configuration') at: 'pharo-eda')
        to: ((ctx at: 'configuration') at: (spec at: 'output
module')) ]]).
    handler handleCommand: command within: Dictionary new

```

And `copySpec:to:` becomes `copySpec:from:to::`

```

{ #category : 'command checking' }
EDAMMMigrationTool >> copySpec: aSpecDictionary from:
    theSourceFolderAsString to: aFolderPathAsString
| result command context |
command := Dictionary new
add: 'class' -> (aSpecDictionary at: 'input class');
add: 'package' -> (aSpecDictionary at: 'input package');
add: 'source root' -> theSourceFolderAsString asFileReference;
yourself.
context := Dictionary new.
result := (EDAMMClassFileReaderCommandHandler new
    handleCommand: command
    within: context)
ifTrue: [ true ]
ifFalse: [ EDAMMClassFileWriterCommandHandler new
    handleCommand:
        (Dictionary new
            add: 'class' -> (aSpecDictionary at: 'input class');
            add:
                'target folder'
                ->
                    (aFolderPathAsString asFileReference
                        / (aSpecDictionary at: 'output package'));
            addAll: context;
            yourself)
        within: context ].
^ result

```

That would only fix one of the problems. `EDAMMMigrationTool` still assumes the input classes come from the `//pharo-eda// "module"`. Currently, that's the only option we have, because that assumption comes from the very definition of the specifications file. To fix it, we need to add a new field to that format, the source module.

10.4 Fixing the specification format

We have a couple of tests in `EDAMMSpecFileTest` to modify. Let's update `testFileReadWithOneRow` first.

```

{ #category : 'tests' }
EDAMMSpecFileTest >> testFileReadWithOneRow
| file entries |
file := FileSystem workingDirectory / 'tests'
/ 'EDAMMSpecFileTest_testFileReadWithOneRow.csv'.
file exists
  ifTrue: [ file delete ].
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDA-API-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: Character cr;
      nextPut: Character lf ].
entries := EDAMMSpecFile new readFile: file.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
  'pharo-eda'.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
  'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
  assert: (entries first at: 'input class')
  equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
  assert: (entries first at: 'output module')
  equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
  assert: (entries first at: 'output package')
  equals: 'EDA-API-Events'.
self assert: (entries first includesKey: 'output class').
self
  assert: (entries first at: 'output class')
  equals: 'EDAEventSourcingEvent'

```

We added a new field, and expected to be parsed as "input module". Fortu-

nately, the fix is very simple.

```
{ #category : 'command checking' }
EDAMMSpecFile >> readFile: aFileReference
| result |
result := OrderedCollection new.
aFileReference
  readStreamDo: [ :stream |
    [ stream atEnd ]
    whileFalse: [ | row |
      row := ',' split: stream nextLine.
      result
        add:
          (Dictionary new
            add: 'input module' -> row first;
            add: 'input package' -> row second;
            add: 'input class' -> row third;
            add: 'output module' -> row fourth;
            add: 'output package' -> row fifth;
            add: 'output class' -> row sixth;
            yourself) ] ].
    ^ result
```

The `testFileReadWithOneRow` is passing now again. It's time to fix `testFileReadWithTwoRows`.

```
{ #category : 'tests' }
EDAMMSpecFileTest >> testFileReadWithTwoRows
| file entries |
file := FileSystem workingDirectory / 'tests'
/ 'EDAMMSpecFileTest_testFileReadWithTwoRows.csv'.
file exists
  ifTrue: [ file delete ].
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDA-API-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: Character cr;
      nextPut: Character lf;
      nextPutAll: 'pharo-eda';
      nextPut: $,;
```

```

        nextPutAll: 'EDA-Voyage-EventStore';
        nextPut: $,;
        nextPutAll: 'EDAMongoEventStore';
        nextPut: $,;
        nextPutAll: 'pharo-eda-adapters';
        nextPut: $,;
        nextPutAll: 'EDAA-EventStore-Voyage';
        nextPut: $,;
        nextPutAll: 'EDAAMongoEventStore';
        nextPut: Character cr;
        nextPut: Character lf ].
entries := EDAMMSpecFile new readFile: file.
self assert: entries isNotNil.
self assert: entries isEmpty.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
    'pharo-eda'.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
    assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
    assert: (entries first at: 'output package')
    equals: 'EDAAPI-Events'.
self assert: (entries first includesKey: 'output class').
self
    assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'.
self assert: entries size > 1.
self assert: (entries second includesKey: 'input module').
self assert: (entries second at: 'input module') equals:
    'pharo-eda'.
self assert: (entries second includesKey: 'input package').
self
    assert: (entries second at: 'input package')
    equals: 'EDA-Voyage-EventStore'.
self assert: (entries second includesKey: 'input class').
self
    assert: (entries second at: 'input class')
    equals: 'EDAMongoEventStore'.
self assert: (entries first includesKey: 'output module').
self

```

```

    assert: (entries second at: 'output module')
    equals: 'pharo-eda-adapters'.
self assert: (entries second includesKey: 'output package').
self
    assert: (entries second at: 'output package')
    equals: 'EDAA-EventStore-Voyage'.
self assert: (entries second includesKey: 'output class').
self
    assert: (entries second at: 'output class')
    equals: 'EDAAMongoEventStore'

```

The next test is in `EDAMMSpecReaderCommandHandlerTest`.

```

{ #category : 'tests' }
EDAMMSpecReaderCommandHandlerTest >> testSpecReaderCommandHandler
| handler command context file entries |
file := FileSystem workingDirectory / 'tests'
/
'EDAMMSpecReaderCommandHandlerTest_testSpecReaderCommandHandler.csv'.
file exists
ifTrue: [ file delete ].
file asFileReference
    writeStreamDo: [ :stream |
        stream
            nextPutAll: 'pharo-eda';
            nextPut: $,;
            nextPutAll: 'EDA-Events';
            nextPut: $,;
            nextPutAll: 'EDAEventSourcingEvent';
            nextPut: $,;
            nextPutAll: 'pharo-eda-api';
            nextPut: $,;
            nextPutAll: 'EDAAPI-Events';
            nextPut: $,;
            nextPutAll: 'EDAEventSourcingEvent';
            nextPut: Character cr;
            nextPut: Character lf ].
context := Dictionary new.
command := Dictionary new
    add: 'specs file' -> file;
    yourself.
handler := EDAMMSpecReaderCommandHandler new.
handler handleCommand: command within: context.
self assert: (context includesKey: 'specs').
entries := context at: 'specs'.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
    'pharo-eda'.
self assert: (entries first includesKey: 'input package').

```

```

self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
    assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
    assert: (entries first at: 'output package')
    equals: 'EDAAPI-Events'.
self assert: (entries first includesKey: 'output class').
self
    assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'

```

Another test we need to change is `EDAMMSpecIteratorCommandHandlerTest#test-SpecIteratorCommandHandler`. After our change, it becomes:

```

{ #category : 'command checking' }
EDAMMSpecIteratorCommandHandlerTest >> testSpecIteratorCommandHandler
| handler command file specsFound |
file := FileSystem workingDirectory / 'tests'
/

'EDAMMSpecIteratorCommandHandlerTest_testSpecIteratorCommandHandler.csv'.
file exists
ifTrue: [ file delete ].
file asFileReference
    writeStreamDo: [ :stream |
        stream
            nextPutAll: 'pharo-eda';
            nextPut: $,;
            nextPutAll: 'EDA-Events';
            nextPut: $,;
            nextPutAll: 'EDAEventSourcingEvent';
            nextPut: $,;
            nextPutAll: 'pharo-eda-api';
            nextPut: $,;
            nextPutAll: 'EDAAPI-Events';
            nextPut: $,;
            nextPutAll: 'EDAEventSourcingEvent';
            nextPut: Character cr;
            nextPut: Character lf;
            nextPutAll: 'pharo-eda';
            nextPut: $,;
            nextPutAll: 'EDA-Domain';

```

```

        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Aggregates';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: Character cr;
        nextPut: Character lf ].
command := Dictionary new
    add: 'specs file' -> file;
    yourself.
specsFound := OrderedCollection new.
handler := EDAMMSpecIteratorCommandHandler
    for: [ :spec :cmd :ctx | specsFound add: spec ].
handler handleCommand: command within: Dictionary new.
self assert: specsFound isEmpty.
self assert: specsFound size equals: 2.
self
    assert: specsFound first
    equals:
        (Dictionary new
            add: 'input module' -> 'pharo-eda';
            add: 'input package' -> 'EDA-Events';
            add: 'input class' -> 'EDAEventSourcingEvent';
            add: 'output module' -> 'pharo-eda-api';
            add: 'output package' -> 'EDAAPI-Events';
            add: 'output class' -> 'EDAEventSourcingEvent';
            yourself).
self
    assert: specsFound second
    equals:
        (Dictionary new
            add: 'input module' -> 'pharo-eda';
            add: 'input package' -> 'EDA-Domain';
            add: 'input class' -> 'EDAAggregateRoot';
            add: 'output module' -> 'pharo-eda-api';
            add: 'output package' -> 'EDAAPI-Aggregates';
            add: 'output class' -> 'EDAAggregateRoot';
            yourself)

```

The only test pending is EDAMMMigrationTool >> testMigrationTool.

```

{ #category: 'tests' }
EDAMMMigrationToolTest >> testMigrationTool
    | outputFolder confFile specFile |
    outputFolder := FileSystem root / 'tmp' / 'pharo-eda-api' / 'src'.
    self deleteFolder: outputFolder.
    outputFolder ensureCreateDirectory.

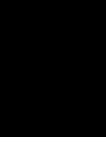
```

```

confFile := FileSystem workingDirectory / 'tests'
/ 'EDAMMMigrationTool_testMigrationTool.config.csv'.
confFile exists
  ifTrue: [ confFile delete ].
confFile asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: (FileSystem workingDirectory / 'src') path
        pathString;
      nextPut: Character cr;
      nextPut: Character lf;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: outputFolder path pathString;
      nextPut: Character cr;
      nextPut: Character lf ].
specFile := FileSystem workingDirectory / 'tests'
/ 'testMigrationTool.spec.csv'.
specFile asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: Character cr;
      nextPut: Character lf;
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Domain';
      nextPut: $,;
      nextPutAll: 'EDAAggregateRoot';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Aggregates';
      nextPut: $,;
      nextPutAll: 'EDAAggregateRoot';
      nextPut: Character cr;
      nextPut: Character lf ].
EDAMMMigrationTool new

```

```
processSpecsFile: specFile
withConfigurationFile: confFile.
self assert: (outputFolder / 'EDAAPI-Events') exists.
self
  assert: (outputFolder / 'EDAAPI-Events' /
    'EDAEventSourcingEvent.st') exists.
self assert: (outputFolder / 'EDAAPI-Aggregates') exists.
self
  assert: (outputFolder / 'EDAAPI-Aggregates' /
    'EDAAggregateRoot.st') exists
```

Changing the package and category of the class

The specification states how a class should be migrated to its new PharoEDA module, and in which package it should reside. So far we've successfully copied classes to their expected submodule, but we leave the package unmodified, regardless of what the specification says.

Our `testMigrationTool` in `EDAMMMigrationToolTest` describes the scenario correctly, but it just doesn't verify anything about the copied classes. We should at least check the package is correct.

Fortunately, we can use our `EDAMMClassFile` class for that.

```
{ #category : 'command checking' }
EDAMMMigrationToolTest >> testMigrationTool
| outputFolder configFile specFile classReader definitions
  classDefinition |
outputFolder := FileSystem root / 'tmp' / 'pharo-eda-api' / 'src'.
self deleteFolder: outputFolder.
outputFolder ensureCreateDirectory.
configFile := FileSystem workingDirectory / 'tests'
              / 'EDAMMMigrationTool_testMigrationTool.config.csv'.
configFile exists
  ifTrue: [ configFile delete ].
configFile asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: (FileSystem workingDirectory / 'src') path
```

```

    pathString;
    nextPut: Character cr;
    nextPut: Character lf;
    nextPutAll: 'pharo-eda-api';
    nextPut: $,;
    nextPutAll: outputFolder path pathString;
    nextPut: Character cr;
    nextPut: Character lf ].
specFile := FileSystem workingDirectory / 'tests'
/ 'testMultimoduleTool.spec.csv'.
specFile asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDA-API-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Domain';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDA-API-Aggregates';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: Character cr;
        nextPut: Character lf ].
EDAMMMigrationTool new
    processSpecsFile: specFile
    withConfigurationFile: confFile.
classReader := EDAMMClassFile new.
self assert: (outputFolder / 'EDA-API-Events') exists.
self
    assert: (outputFolder / 'EDA-API-Events' /
        'EDAEventSourcingEvent.st') exists.
definitions := classReader
    readClassNamed: 'EDAEventSourcingEvent'
    inPackage: 'EDA-API-Events'

```

```

    inFolder: outputFolder.
self
  assert: (definitions select: [ :def | def isClassDefinition ])
    isEmpty.
classDefinition := (definitions
  select: [ :def | def isClassDefinition ]) first.
self assert: classDefinition isNotNil.
self assert: classDefinition category equals: 'EDAAPI-Events'.
self assert: (outputFolder / 'EDAAPI-Aggregates') exists.
self
  assert: (outputFolder / 'EDAAPI-Aggregates' /
    'EDAAggregateRoot.st') exists.
definitions := classReader
  readClassNamed: 'EDAAggregateRoot'
  inPackage: 'EDAAPI-Aggregates'
  inFolder: outputFolder.
self
  assert: (definitions select: [ :def | def isClassDefinition ])
    isEmpty.
classDefinition := (definitions
  select: [ :def | def isClassDefinition ]) first.
self assert: classDefinition isNotNil.
self assert: classDefinition category equals: 'EDAAPI-Aggregates'

```

We've just added two checks to verify the packages are correct. The test fails with "TestFailure: Got #'EDA-Events-Events' instead of 'EDAAPI-Events'". We used `MCClassDefinition >> category` because it's the only information about the package we know.

To clarify this: when using Tonel format, the package of the class is the name of the folder the ".st" is in, but the category of the class needs to be compatible with the name of the package. That is, it either matches the name of the package, or starts with it and appends a suffix (starting with "-").

Strictly speaking, our tool respects the format. However, it doesn't do anything with the category itself, and therefore will lead to inconsistencies if the package differs (because the category won't).

There're two options:

- we try to be super clever and come up with a category by ourselves.
- we update the specification to include the category of the copied class.

The second option is better. First of all, it's explicit, and allows us to implement it faster. Second, it can accommodate the first option as well if we need. We could add a new handler that provides the value of the new category, if the specification omits it, based on the source category, the source package, and the destination package.

11.1 Updating the specification to include the target category

We have `EDAMMSpecReaderCommandHandlerTest`, so let's use it.

```
{ #category : 'command checking' }
EDAMMSpecReaderCommandHandlerTest >> testSpecReaderCommandHandler
| handler command context file entries |
file := FileSystem workingDirectory / 'tests'
/
'EDAMMSpecReaderCommandHandlerTest_testSpecReaderCommandHandler.csv'.
file exists
ifTrue: [ file delete ].
file asFileReference
writeStreamDo: [ :stream |
stream
nextPutAll: 'pharo-eda';
nextPut: $,;
nextPutAll: 'EDA-Events';
nextPut: $,;
nextPutAll: 'EDAEventSourcingEvent';
nextPut: $,;
nextPutAll: 'pharo-eda-api';
nextPut: $,;
nextPutAll: 'EDA-API-Events';
nextPut: $,;
nextPutAll: 'EDA-API-Events-Events';
nextPut: $,;
nextPutAll: 'EDAEventSourcingEvent';
nextPut: Character cr;
nextPut: Character lf ].
context := Dictionary new.
command := Dictionary new
add: 'specs file' -> file;
yourself.
handler := EDAMMSpecReaderCommandHandler new.
handler handleCommand: command within: context.
self assert: (context includesKey: 'specs').
entries := context at: 'specs'.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
'pharo-eda'.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
assert: (entries first at: 'input class')
```

11.1 Updating the specification to include the target category

```
    equals: 'EDAEventSourcingEvent'.
  self assert: (entries first includesKey: 'output module').
  self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
  self assert: (entries first includesKey: 'output package').
  self
    assert: (entries first at: 'output package')
    equals: 'EDAAPI-Events'.
  self assert: (entries first includesKey: 'output category').
  self
    assert: (entries first at: 'output category')
    equals: 'EDAAPI-Events-Events'.
  self assert: (entries first includesKey: 'output class').
  self
    assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'
```

We just added the new field and the associated asserts.

To make it pass, we have to change EDAMMSpecFile.

```
{ #category : 'reading' }
EDAMMSpecFile >> readFile: aFileReference
| result |
result := OrderedCollection new.
aFileReference
  readStreamDo: [ :stream |
    [ stream atEnd ]
    whileFalse: [ | row |
      row := ', ' split: stream nextLine.
      result
        add:
          (Dictionary new
            add: 'input module' -> row first;
            add: 'input package' -> row second;
            add: 'input class' -> row third;
            add: 'output module' -> row fourth;
            add: 'output package' -> row fifth;
            add: 'output category' -> row sixth;
            add: 'output class' -> row seventh;
            yourself) ] ].
    ^ result
```

The test passes again.

We have to modify EDAMMMigrationToolTest >> testMigrationTool to accomodate the new spec format.

We'll split the test into different methods, following the BDD style, to make it cleaner.

```
{ #category : 'tests' }
EDAMMigrationToolTest >> testMigrationTool
self _testMigrationTool_example
```

The example method composes the "given", "when" and "then" parts.

```
{ #category : 'test support' }
EDAMMigrationToolTest >> _testMigrationTool_example
| given when then |
given := self _testMigrationTool_given.
when := [ :specFile :confFile | self
_testMigrationTool_when_specFile: specFile confFile: confFile ].
then := [ self _testMigrationTool_then ].
^ Dictionary new
add: #given -> given;
add:
    #when
    -> (when value: (given at: #specFile) value: (given at:
    #confFile));
add: #then -> then value;
yourself
```

The "given" creates the output folder, the spec and the configuration files.

```
{ #category : 'test support' }
EDAMMigrationToolTest >> _testMigrationTool_given
| outputFolder confFile specFile |
outputFolder := FileSystem root / 'tmp' / 'pharo-eda-api' / 'src'.
self deleteFolder: outputFolder.
outputFolder ensureCreateDirectory.
confFile := FileSystem workingDirectory / 'tests'
/ 'EDAMMigrationTool_testMigrationTool.config.csv'.
confFile exists
ifTrue: [ confFile delete ].
confFile asFileReference
writeStreamDo: [ :stream |
stream
nextPutAll: 'pharo-eda';
nextPut: $,;
nextPutAll: (FileSystem workingDirectory / 'src') path
pathString;
nextPut: Character cr;
nextPut: Character lf;
nextPutAll: 'pharo-eda-api';
nextPut: $,;
nextPutAll: outputFolder path pathString;
nextPut: Character cr;
nextPut: Character lf ].
specFile := FileSystem workingDirectory / 'tests'
/ 'testMultimoduleTool.spec.csv'.
specFile asFileReference
```

11.1 Updating the specification to include the target category

```
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Domain';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Aggregates';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Aggregates';
        nextPut: $,;
        nextPutAll: 'EDAAggregateRoot';
        nextPut: Character cr;
        nextPut: Character lf ].
^ Dictionary new
  add: #specFile -> specFile;
  add: #confFile -> confFile;
  yourself
```

The "when" method uses the specification and the configuration files to call EDAMMMigrationTool.

```
{ #category : 'test support' }
EDAMMMigrationToolTest >> _testMigrationTool_when_specFile: specFile
  confFile: confFile
  ^ EDAMMMigrationTool new
    processSpecsFile: specFile
    withConfigurationFile: confFile
```

The "then" contains the assertions.

```

{ #category : 'test support' }
EDAMMMigrationToolTest >> _testMigrationTool_then
| classReader outputFolder definitions classDefinition |
outputFolder := FileSystem root / 'tmp' / 'pharo-eda-api' / 'src'.
classReader := EDAMMClassFile new.
self assert: (outputFolder / 'EDAAPI-Events') exists.
self
  assert: (outputFolder / 'EDAAPI-Events' /
    'EDAEventSourcingEvent.st') exists.
definitions := classReader
  readClassNamed: 'EDAEventSourcingEvent'
  inPackage: 'EDAAPI-Events'
  inFolder: outputFolder.
self
  assert: (definitions select: [ :def | def isClassDefinition ])
    isEmpty.
classDefinition := (definitions
  select: [ :def | def isClassDefinition ]) first.
self assert: classDefinition isNotNil.
self assert: classDefinition category equals:
  'EDAAPI-Events-Events'.
self assert: (outputFolder / 'EDAAPI-Aggregates') exists.
self
  assert: (outputFolder / 'EDAAPI-Aggregates' /
    'EDAAggregateRoot.st') exists.
definitions := classReader
  readClassNamed: 'EDAAggregateRoot'
  inPackage: 'EDAAPI-Aggregates'
  inFolder: outputFolder.
self
  assert: (definitions select: [ :def | def isClassDefinition ])
    isEmpty.
classDefinition := (definitions
  select: [ :def | def isClassDefinition ]) first.
self assert: classDefinition isNotNil.
self assert: classDefinition category equals: 'EDAAPI-Aggregates'

```

Now the test is correct. We don't check the package, because it's derived from the package the ".st" file gets created. But we can perform some checks on the category. And those checks fail, because we are not using the new field in the spec when copying the classes.

There're some other tests we need to update after changing the spec.

First, `testFileReadWithOneRow` in `EDAMMSpecFileTest`:

```

{ #category : 'tests' }
EDAMMSpecFileTest >> testFileReadWithOneRow
| file entries |
file := FileSystem workingDirectory / 'tests'
/ 'EDAMMSpecFileTest_testFileReadWithOneRow.csv'.

```

11.1 Updating the specification to include the target category

```
file exists
  ifTrue: [ file delete ].
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Events';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Events-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: Character cr;
      nextPut: Character lf ].
entries := EDAMMSpecFile new readFile: file.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
  'pharo-eda'.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
  'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
  assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
  assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
  assert: (entries first at: 'output package')
    equals: 'EDAAPI-Events'.
self assert: (entries first includesKey: 'output class').
self
  assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'
```

Second, `testFileReadWithTwoRows` in the same test suite.

```

{ #category : 'tests' }
EDAMMSpecFileTest >> testFileReadWithTwoRows
| file entries |
file := FileSystem workingDirectory / 'tests'
/ 'EDAMMSpecFileTest_testFileReadWithTwoRows.csv'.
file exists
ifTrue: [ file delete ].
file asFileReference
writeStreamDo: [ :stream |
    stream
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: Character cr;
        nextPut: Character lf;
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Voyage-EventStore';
        nextPut: $,;
        nextPutAll: 'EDAMongoEventStore';
        nextPut: $,;
        nextPutAll: 'pharo-eda-adapters';
        nextPut: $,;
        nextPutAll: 'EDAA-EventStore-Voyage';
        nextPut: $,;
        nextPutAll: 'EDAA-EventStore-Voyage';
        nextPut: $,;
        nextPutAll: 'EDAAMongoEventStore';
        nextPut: Character cr;
        nextPut: Character lf ].
entries := EDAMMSpecFile new readFile: file.
self assert: entries isNotNil.
self assert: entries isEmpty.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
    'pharo-eda'.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').

```

11.1 Updating the specification to include the target category

```
self
  assert: (entries first at: 'input class')
  equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
  assert: (entries first at: 'output module')
  equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
  assert: (entries first at: 'output package')
  equals: 'EDAAPI-Events'.
self assert: (entries first includesKey: 'output class').
self
  assert: (entries first at: 'output class')
  equals: 'EDAEventSourcingEvent'.
self assert: entries size > 1.
self assert: (entries second includesKey: 'input module').
self assert: (entries second at: 'input module') equals:
  'pharo-eda'.
self assert: (entries second includesKey: 'input package').
self
  assert: (entries second at: 'input package')
  equals: 'EDA-Voyage-EventStore'.
self assert: (entries second includesKey: 'input class').
self
  assert: (entries second at: 'input class')
  equals: 'EDAMongoEventStore'.
self assert: (entries first includesKey: 'output module').
self
  assert: (entries second at: 'output module')
  equals: 'pharo-eda-adapters'.
self assert: (entries second includesKey: 'output package').
self
  assert: (entries second at: 'output package')
  equals: 'EDAA-EventStore-Voyage'.
self assert: (entries second includesKey: 'output class').
self
  assert: (entries second at: 'output class')
  equals: 'EDAAMongoEventStore'
```

Third, testSpecIteratorCommandHandler in EDAMMSpecIteratorCommandHandlerTest.

```
{ #category : 'tests' }
EDAMMSpecIteratorCommandHandlerTest >> testSpecIteratorCommandHandler
"tests"
testSpecIteratorCommandHandler
| handler command file specsFound |
file := FileSystem workingDirectory / 'tests'
/
```

```

'EDAMMSpecIteratorCommandHandlerTest_testSpecIteratorCommandHandler.csv'.
file exists
  ifTrue: [ file delete ].
file asFileReference
  writeStreamDo: [ :stream |
    stream
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Events';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Events-Events';
      nextPut: $,;
      nextPutAll: 'EDAEventSourcingEvent';
      nextPut: Character cr;
      nextPut: Character lf;
      nextPutAll: 'pharo-eda';
      nextPut: $,;
      nextPutAll: 'EDA-Domain';
      nextPut: $,;
      nextPutAll: 'EDAAggregateRoot';
      nextPut: $,;
      nextPutAll: 'pharo-eda-api';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Aggregates';
      nextPut: $,;
      nextPutAll: 'EDAAPI-Aggregates';
      nextPut: $,;
      nextPutAll: 'EDAAggregateRoot';
      nextPut: Character cr;
      nextPut: Character lf ].
command := Dictionary new
  add: 'specs file' -> file;
  yourself.
specsFound := OrderedCollection new.
handler := EDAMMSpecIteratorCommandHandler
  for: [ :spec :cmd :ctx | specsFound add: spec ].
handler handleCommand: command within: Dictionary new.
self assert: specsFound isEmpty.
self assert: specsFound size equals: 2.
self
  assert: specsFound first
  equals:
    (Dictionary new

```

11.1 Updating the specification to include the target category

```
add: 'input module' -> 'pharo-eda';
add: 'input package' -> 'EDA-Events';
add: 'input class' -> 'EDAEventSourcingEvent';
add: 'output module' -> 'pharo-eda-api';
add: 'output package' -> 'EDAAPI-Events';
add: 'output category' -> 'EDAAPI-Events-Events';
add: 'output class' -> 'EDAEventSourcingEvent';
yourself).

self
  assert: specsFound second
  equals:
    (Dictionary new
      add: 'input module' -> 'pharo-eda';
      add: 'input package' -> 'EDA-Domain';
      add: 'input class' -> 'EDAAggregateRoot';
      add: 'output module' -> 'pharo-eda-api';
      add: 'output package' -> 'EDAAPI-Aggregates';
      add: 'output category' -> 'EDAAPI-Aggregates';
      add: 'output class' -> 'EDAAggregateRoot';
      yourself)
```

Finally, `testSpecReaderCommandHandler` in `EDAMMSpecReaderCommandHandlerTest`:

```
{ #category : 'tests' }
EDAMMSpecReaderCommandHandlerTest >> testSpecReaderCommandHandler
"tests"
testSpecReaderCommandHandler
  | handler command context file entries |
  file := FileSystem workingDirectory / 'tests'
  /
    'EDAMMSpecReaderCommandHandlerTest_testSpecReaderCommandHandler.csv'.
  file exists
    ifTrue: [ file delete ].
  file asFileReference
    writeStreamDo: [ :stream |
      stream
        nextPutAll: 'pharo-eda';
        nextPut: $,;
        nextPutAll: 'EDA-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
        nextPut: $,;
        nextPutAll: 'pharo-eda-api';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events';
        nextPut: $,;
        nextPutAll: 'EDAAPI-Events-Events';
        nextPut: $,;
        nextPutAll: 'EDAEventSourcingEvent';
```

```

        nextPut: Character cr;
        nextPut: Character lf ].
context := Dictionary new.
command := Dictionary new
    add: 'specs file' -> file;
    yourself.
handler := EDAMMSpecReaderCommandHandler new.
handler handleCommand: command within: context.
self assert: (context includesKey: 'specs').
entries := context at: 'specs'.
self assert: entries isNotNil.
self assert: (entries first includesKey: 'input module').
self assert: (entries first at: 'input module') equals:
    'pharo-eda'.
self assert: (entries first includesKey: 'input package').
self assert: (entries first at: 'input package') equals:
    'EDA-Events'.
self assert: (entries first includesKey: 'input class').
self
    assert: (entries first at: 'input class')
    equals: 'EDAEventSourcingEvent'.
self assert: (entries first includesKey: 'output module').
self
    assert: (entries first at: 'output module')
    equals: 'pharo-eda-api'.
self assert: (entries first includesKey: 'output package').
self
    assert: (entries first at: 'output package')
    equals: 'EDA-API-Events'.
self assert: (entries first includesKey: 'output category').
self
    assert: (entries first at: 'output category')
    equals: 'EDA-API-Events-Events'.
self assert: (entries first includesKey: 'output class').
self
    assert: (entries first at: 'output class')
    equals: 'EDAEventSourcingEvent'

```

11.2 New handler for overwriting the category

The classes responsible of reading and writing classes in Tonel format are `EDAMMClassFileReaderCommandHandler` and `EDAMMClassFileWriterCommandHandler`. The category information is just data from the class definition. If we changed them to deal with this category issue, we'd be adding a responsibility they don't need. After all, we are using a chain of responsibility, and that pattern is designed precisely to help us adding new responsibilities without needing to modify existing ones. We should add a new command handler in between the two, that replaces the original class definition with

another one, that merely changes the category according to the spec.

Let's follow that line of thought, and write an initial test for the new command handler.

```
TestCase subclass:
    #EDAMMClassFileCategoryOverwritingCommandHandlerTest
    instanceVariableNames: ''
    classVariableNames: ''
    package: 'EDA-Multimodule-Migration-Tests'
```

The test is based on `EDAMMClassFileReaderCommandHandlerTest`'s `test-ClassFileReaderCommandHandler`:

```
{ #category : 'tests' }
EDAMMClassFileCatalogOverwritingCommandHandlerTest >>
    testClassFileCatalogOverwritingCommandHandler
    | handler command context definitions |
    command := Dictionary new
        add: 'class' -> 'EDACommandConsumer';
        add: 'package' -> 'EDA-Messaging';
        add: 'source root' -> (FileSystem workingDirectory / 'src');
        yourself.
    context := Dictionary new.
    handler := EDAMMCompositeCommandHandler
        with: EDAMMClassFileReaderCommandHandler new
        with:
            (EDAMMClassFileCategoryOverwritingCommandHandler
                withCatalog: 'EDA-Messaging-Command-Overwritten').
    handler handleCommand: command within: context.
    self assert: (context includesKey: 'EDACommandConsumer').
    definitions := context at: 'EDACommandConsumer'.
    self assert: definitions isNotNil.
    self assert: definitions isEmpty.
    definitions
        select: [ :def | def isClassDefinition ]
        thenCollect:
            [ :def | self assert: def category equals:
                'EDA-Messaging-Command-Overwritten' ]
```

We need to create the `EDAMMClassFileCategoryOverwritingCommandHandler` class.

```
EDAMMCommandHandler subclass:
    #EDAMMClassFileCategoryOverwritingCommandHandler
    instanceVariableNames: 'category'
    classVariableNames: ''
    package: 'EDA-Multimodule-Migration'
```

Let's not forget the accessors for category as well.

```
[ { #category : 'accessing' }
category
^ category

{ #category : 'accessing' }
category: aString
category := aString
```

The test expects `EDAMMClassFileCategoryOverwritingCommandHandler` to provide a factory method.

```
[ { #category : 'instance creation' }
withCatalog: aString
^ self new
category: aString;
yourself
```

Running the test now reports a `shouldBeImplemented` signal. It's time to implement `handleCommand:within:` in `EDAMMClassFileCategoryOverwritingCommandHandler`.

We'll wrap the original `MCClassDefinition` instance with a new instance, whose only purpose is to intercept the `catalog` message and return the overwritten value.

```
[ { #category : 'command handling' }
EDAMMClassFileCategoryOverwritingCommandHandler >> handleCommand:
aCommand within: aContext
| result |
result := false.
(self inputValid: aCommand within: aContext)
ifTrue: [ | classDefinition definitions newDefinitions className
|
className := aCommand at: 'class'.
definitions := aContext at: className.
classDefinition := (definitions
select: [ :def | def isClassDefinition ]) first.
newDefinitions := OrderedCollection
with:
(EDAMMClassDefinitionWrapper
for: classDefinition
andCategory: self category).
newDefinitions
addAll: (definitions reject: [ :def | def isClassDefinition
]).
aContext at: className put: newDefinitions ].
^ result
```

Saving the test code will allow us to create the new class `EDAMMClassDefinitionWrapper`:

```
Object subclass: #EDAMMClassDefinitionWrapper
  instanceVariableNames: 'wrapped category'
  classVariableNames: ''
  package: 'EDA-Multimodule-Migration'
```

After creating the accessors, we can deal with the important part of this class.

```
{ #category : 'accessing' }
EDAMMClassDefinitionWrapper >> category
^ category

{ #category : 'accessing' }
EDAMMClassDefinitionWrapper >> category: aString
category := aString

{ #category : 'accessing' }
EDAMMClassDefinitionWrapper >> wrapped
^ wrapped

{ #category : 'accessing' }
EDAMMClassDefinitionWrapper >> wrapped: anMCClassDefinition
wrapped := anMCClassDefinition
```

We'll make use of the reflective capabilities of Smalltalk. Instead of letting the Object class handle unimplemented messages, we'll overwrite `doesNotUnderstand:` to delegate them to the wrapped instance.

```
{ #category : 'reflective operations' }
EDAMMClassDefinitionWrapper >> doesNotUnderstand: aMessage
^ aMessage hasArguments
  ifTrue: [ self wrapped perform: aMessage selector with: aMessage
    arguments ]
  ifFalse: [ self wrapped perform: aMessage selector ]
```

The only missing piece is the guard checking we included in `handleCommand:within:`.

```
{ #category : 'command checking' }
EDAMMClassFileCategoryOverwritingCommandHandlerTest >> inputValid:
  aCommand within: aContext
  | result |
  result := aCommand includesKey: 'class'.
  result
    ifTrue: [ | className |
      className := aCommand at: 'class'.
      result := className isNotNil
        and: [ aContext includesKey: className ].
      result
        ifTrue: [ | definitions classDefinitions |
          definitions := aContext at: className.
          classDefinitions := definitions
            select: [ :def | def isClassDefinition ].
```

```

    result := classDefinitions isEmpty ] ].
  ^ result

```

The only thing left to do is to include this handler as part of our EDAMMMigrationTool, which is the same as to say to make testMigrationTool test of EDAMMMigrationToolTest pass.

The change we need to make is in EDAMMMigrationTool's copySpec:from:to:.

```

{ #category : 'copying' }
EDAMMMigrationTool >> copySpec: aSpecDictionary from:
    theSourceFolderAsString to: aFolderPathAsString
| result command context |
command := Dictionary new
    add: 'class' -> (aSpecDictionary at: 'input class');
    add: 'package' -> (aSpecDictionary at: 'input package');
    add: 'source root' -> theSourceFolderAsString asFileReference;
    yourself.
context := Dictionary new.
result := (EDAMMClassFileReaderCommandHandler new
    handleCommand: command
    within: context)
    ifTrue: [ true ]
    ifFalse: [ (EDAMMCompositeCommandHandler
        with:
            (EDAMMClassFileCategoryOverwritingCommandHandler
                withCatalog: (aSpecDictionary at: 'output category'))
            with: EDAMMClassFileWriterCommandHandler new)
        handleCommand:
            (Dictionary new
                add: 'class' -> (aSpecDictionary at: 'input class');
                add:
                    'target folder'
                    ->
                        (aFolderPathAsString asFileReference
                            / (aSpecDictionary at: 'output package'));
                addAll: context;
                yourself)
            within: context ].
    ^ result

```

There're two problems. First, we're using handlers in an isolate manner. We're not using a chain of responsibility, and thus we're not leveraging its benefits. Second, it doesn't work. If you run EDAMMMigrationToolTest >> testMigrationTool, you'll see that, even though our new EDAMMClassFileCategoryOverwritingCommandHandler gets executed, it makes no difference.

The cause is in EDAMMClassFileWriterCommandHandler's handleCommand:within::

```

EDAMMClassFileWriterCommandHandler >> handleCommand: aCommand
  within: aContext
  | result |
  result := false.
  (self inputValid: aCommand)
    ifTrue: [ | className |
      className := aCommand at: 'class'.
      EDAMMClassFile new
        writeClassName: className
        withDefinitions: (aCommand at: className)
        inFolder: (aCommand at: 'target folder') ].
    ^ result

```

EDAMMClassFileWriterCommandHandler takes the class definitions from the command, whereas EDAMMClassFileCategoryOverwritingCommandHandler's output is stored in the context instead.

Let's try to make the test pass the easiest way possible from our current situation, and delay the refactoring until then.

There're two things. First, EDAMMClassFileCategoryOverwritingCommandHandler transforms the class definitions and stores them in the context. However, EDAMMClassFileWriterCommandHandler reads its input from the command instead. We'll refactor this later, but meanwhile the simplest fix is to overwrite the contents of the command:

```

{ #category : 'command handling' }
EDAMMClassFileCategoryOverwritingCommandHandler >> handleCommand:
  aCommand within: aContext
  | result |
  result := false.
  (self inputValid: aCommand within: aContext)
    ifTrue: [ | classDefinition definitions newDefinitions className |
      className := aCommand at: 'class'.
      definitions := aContext at: className.
      classDefinition := (definitions
        select: [ :def | def isClassDefinition ]) first.
      newDefinitions := OrderedCollection
        with:
          (EDAMMClassDefinitionWrapper
            for: classDefinition
            andCategory: self category).
      newDefinitions
        addAll: (definitions reject: [ :def | def isClassDefinition
        ]).
      aCommand at: className put: newDefinitions ].
  ^ result

```

The next problem is that TonelWriter performs a (questionable) type check

on the definitions, to check which one defines the class:

```
{ #category : 'private testing' }
Tone1Writer >> isClass: aClassDefinition
^ aClassDefinition class = MCClassDefinition
```

We could use an extension method to overwrite that method, but a less intrusive option would be to overload the class method on EDAMMClassDefinitionWrapper.

```
{ #category : 'class membership' }
EDAMMClassDefinitionWrapper >> class
^ self wrapped class
```

The final problem is in EDAMMClassFile. When writing the definitions, it uses classDefinition className to filter definitions actually belonging to the class. However, classDefinition is actually EDAMMClassDefinitionWrapper, so it's actually filtering out all definitions. Again, the simplest fix is to overload className in EDAMMClassDefinitionWrapper:

```
{ #category : 'introspection' }
className
^ self wrapped className
```

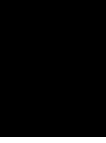
Now we should have the test passing. However, we broke EDAMMClassFileCatalogOverwritingCommandHandlerTest's testClassFileCatalogOverwritingCommandHandler in the process of storing the definitions in the command instead of the context. The fix is simple.

```
{ #category : 'tests' }
EDAMMClassFileCatalogOverwritingCommandHandlerTest >>
    testClassFileCatalogOverwritingCommandHandler
| handler command context definitions |
command := Dictionary new
    add: 'class' -> 'EDACommandConsumer';
    add: 'package' -> 'EDA-Messaging';
    add: 'source root' -> (FileSystem workingDirectory / 'src');
yourself.
context := Dictionary new.
handler := EDAMMCompositeCommandHandler
    with: EDAMMClassFileReaderCommandHandler new
    with:
        (EDAMMClassFileCategoryOverwritingCommandHandler
            withCatalog: 'EDA-Messaging-Command-Overwritten').
handler handleCommand: command within: context.
self assert: (command includesKey: 'EDACommandConsumer').
definitions := command at: 'EDACommandConsumer'.
self assert: definitions isNotNil.
self assert: definitions isNotEmpty.
definitions
    select: [ :def | def isClassDefinition ]
```

11.2 New handler for overwriting the category

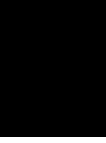
```
i
  thenCollect:
    [ :def | self assert: def category equals:
      'EDA-Messaging-Command-Overwritten' ]
```

Now all tests are in green, and we're able to overwrite the package and category of the classes we copy.



Renaming references

We are now able to copy classes from one package in a module, to a different package in a different module. However, since the classes can be renamed in the process, we need to make sure references get updated too.



Writing the specs

Finally we can profit from our modularization. Recall our journey started with a problem statement: automate copying and renaming classes from `pharo-eda develop`, to the new `pharo-eda` modules.

The specification format was defined in Chapter04:

```
[ input package, input class, output module, output package, output class
  EDA-Commands, EDACommand, pharo-eda-api, EDAAPI-Commands, EDAAPI-Commands, EDACommand
  EDA-Commands, EDADeleteAggregateCommand, pharo-eda-api, EDAAPI-Commands, EDAAPI-Comm
  EDA-Commands, EDAUserCommand, pharo-eda-api, EDAAPI-Commands, EDAAPI-Commands, EDAUserC
  EDA-Commands, EDACommandError, pharo-eda-api, EDAAPI-Commands, EDAAPI-Commands, EDAComm
  EDA-Commands, EDATooManyAggregatesForCriteria, EDAAPI-Commands, EDAAPI-Commands, EDATo
  EDA-Commands-Tests, RegexJson, pharo-eda, EDA-Commands-Tests, EDA-Commands-Tests, EDARE
  EDA-Commands-Tests, RegexJsonArray, pharo-eda, EDA-Commands-Tests, EDAREgexJsonArray
  EDA-Commands-Tests, RegexJsonDictionary, pharo-eda, EDA-Commands-Tests, EDA-Commands-T
  EDA-Commands-Tests, EDATestSettings, pharo-eda-settings, EDAS-Settings-Tests, EDAS-Set
  EDA-Domain, EDAAggregateRepository, pharo-eda-api, EDAAPI-Aggregates, EDAAPI-Aggregate
  EDA-Domain, EDAAggregateRoot, pharo-eda-api, EDAAPI-Aggregates, EDAAPI-Aggregates, EDAA
  EDA-Domain, EDADomainException, pharo-eda-api, EDAAPI-Exceptions, EDAAPI-Exceptions, ED
  EDA-Domain, EDAAggregateRootDoesNotExist, pharo-eda-api, EDAAPI-Exceptions, EDAAPI-Exc
  EDA-Domain, EDAAlreadyExistingAggregate, pharo-eda-api, EDAAPI-Exceptions, EDAAPI-Exce
  EDA-Domain, MissingDeleteEventClassForAggregate, pharo-eda-api, EDAAPI-Exceptions, EDA
  EDA-Domain, TooManyDeleteEventClassesForAggregate, pharo-eda-api, EDAAPI-Exceptions, E
  EDA-Domain, InvalidTenant, pharo-eda-api, EDAAPI-Exceptions, EDAAPI-Exceptions, EDInva
  EDA-Domain-Tests, EDAAcceptEverything, pharo-eda, EDA-TestSupport, EDA-TestSupport, EDA
  EDA-Domain-Tests, EDATDomainBDD, pharo-eda, EDA-TestSupport, EDA-TestSupport, EDATDomain
  EDA-Domain-Tests, EDATTestEvent, pharo-eda, EDA-TestSupport, EDA-TestSupport, EDATestEv
  EDA-Domain-Tests, EDATestAggregateRoot, pharo-eda, EDA-TestSupport, EDA-TestSupport, ED
  EDA-Domain-Tests, EDATestEventStub, pharo-eda, EDA-TestSupport, EDA-TestSupport, EDATes
  EDA-Domain-Tests, EDAVolatileEventStore, pharo-eda, EDA-TestSupport, EDA-TestSupport, E
```

EDA-Errors, EDAErrorBehavior, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Behaviors, EDAError
 EDA-Errors, EDACommandConsumingErrorBehavior, pharo-eda-errors, EDAE-Domain, EDAE-Domain-
 EDA-Errors, EDAProjectionErrorBehavior, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Behavi
 EDA-Errors, EDAErrorHandlingStrategy, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Behavior
 EDA-Errors, EDAJustLogErrorStrategy, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Behaviors
 EDA-Errors, EDALaunchDebuggerErrorStrategy, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Be
 EDA-Errors, EDAPublishMessageErrorStrategy, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Be
 EDA-Errors, EDAThrowError, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Behaviors, EDAThrow
 EDA-Errors, EDAJsonErrorSerializer, pharo-eda-errors, EDAE-Domain, EDAE-Domain-Behaviors,
 EDA-EventStore, EDAEventStore, pharo-eda-ports, EDAP-EventStore, EDAP-EventStore, EDAEvent
 EDA-Events, EDAEventSourcingEvent, pharo-eda-api, EDAAPI-Events, EDAAPI-Events, EDAEventSo
 EDA-Events, EDACommandResultEvent, pharo-eda-api, EDAAPI-Events, EDAAPI-Events, EDACommand
 EDA-Events, EDADeleteCommandResultEvent, pharo-eda-api, EDAAPI-Events, EDAAPI-Events, EDAD
 EDA-Events, EDAUserCommandResultEvent, pharo-eda-api, EDAAPI-Events, EDAAPI-Events, EDAUse
 EDA-Events, EDAVersionedEvent, pharo-eda-api, EDAAPI-Events, EDAAPI-Events, EDAVersionedEv
 EDA-Logging, EDALogEvent, pharo-eda-api, EDAAPI-Logging, EDAAPI-Logging, EDALogEvent
 EDA-Logging, InvalidLoggingLevel, pharo-eda-common, EDAC-Logging, EDAC-Logging, EDACInvali
 EDA-Messaging, EDACommandConsumer, pharo-eda-ports, EDAP-Messaging, EDAP-Messaging-Comman
 EDA-Messaging, EDAEventProducer, pharo-eda-ports, EDAP-Messaging, EDAP-Messaging-Event, ED
 EDA-Messaging, EDAMessageSerializer, pharo-eda-common, EDAC-Messaging, EDAC-Messaging, ED
 EDA-Messaging, EDATopicListener, pharo-eda-ports, EDAP-Messaging, EDAP-Messaging-Topic, ED
 EDA-Messaging, EDATopicPublisher, pharo-eda-ports, EDAP-Messaging, EDAP-Messaging-Topic, E
 EDA-Multimodule-Migration, EDAMMClassDefinitionWrapper, pharo-eda-tools, EDAT-Tools, EDAT
 EDA-Multimodule-Migration, EDAMMClassFile, pharo-eda-tools, EDAT-Tools, EDAT-Tools-Migrat
 EDA-Multimodule-Migration, EDAMMCommandHandler, pharo-eda-tools, EDAT-Tools, EDAT-Tools-M
 EDA-Multimodule-Migration, EDAMMClassFileCategoryOverwritingCommandHandler, pharo-eda-t
 EDA-Multimodule-Migration, EDAMMClassFileReaderCommandHandler, pharo-eda-tools, EDAT-Too
 EDA-Multimodule-Migration, EDAMMClassFileWriterCommandHandler, pharo-eda-tools, EDAT-Too
 EDA-Multimodule-Migration, EDAMMCompositeCommandHandler, pharo-eda-tools, EDAT-Tools, EDA
 EDA-Multimodule-Migration, EDAMMConfigurationCommandHandler, pharo-eda-tools, EDAT-Tools
 EDA-Multimodule-Migration, EDAMMLoggingCommandHandler, pharo-eda-tools, EDAT-Tools, EDAT
 EDA-Multimodule-Migration, EDAMMSpecIteratorCommandHandler, pharo-eda-tools, EDAT-Tools,
 EDA-Multimodule-Migration, EDAMMSpecReaderCommandHandler, pharo-eda-tools, EDAT-Tools, ED
 EDA-Multimodule-Migration, EDAMMMigrationTool, pharo-eda-tools, EDAT-Tools, EDAT-Tools-Mi
 EDA-Multimodule-Migration, EDAMMModuleFile, pharo-eda-tools, EDAT-Tools, EDAT-Tools-Migra
 EDA-Multimodule-Migration, EDAMMSpecFile, pharo-eda-tools, EDAT-Tools, EDAT-Tools-Migrati
 EDA-Multimodule-Migration-Tests, EDAMMClassFileCategoryOverwritingCommandHandlerTest, p
 EDA-Multimodule-Migration-Tests, EDAMMClassFileReaderCommandHandlerTest, pharo-eda-tool
 EDA-Multimodule-Migration-Tests, EDAMMClassFileTest, pharo-eda-tools, EDAT-Tools, EDAT-To
 EDA-Multimodule-Migration-Tests, EDAMMClassFileWriterCommandHandlerTest, pharo-eda-tool
 EDA-Multimodule-Migration-Tests, EDAMMCommandHandlerTest, pharo-eda-tools, EDAT-Tools, ED
 EDA-Multimodule-Migration-Tests, EDAMMCompositeCommandHandlerTest, pharo-eda-tools, EDAT
 EDA-Multimodule-Migration-Tests, EDAMMConfigurationCommandHandlerTest, pharo-eda-tools,
 EDA-Multimodule-Migration-Tests, EDAMMDummyCommandHandler, pharo-eda-tools, EDAT-Tools, E
 EDA-Multimodule-Migration-Tests, EDAMMMigrationToolTest, pharo-eda-tools, EDAT-Tools, EDA
 EDA-Multimodule-Migration-Tests, EDAMMModuleFileTest, pharo-eda-tools, EDAT-Tools, EDAT-T
 EDA-Multimodule-Migration-Tests, EDAMMSpecFileTest, pharo-eda-tools, EDAT-Tools, EDAT-Too
 EDA-Multimodule-Migration-Tests, EDAMMSpecIteratorCommandHandlerTest, pharo-eda-tools, E
 EDA-Multimodule-Migration-Tests, EDAMMSpecReaderCommandHandlerTest, pharo-eda-tools, EDA
 EDA-Projections, EDAProjection, pharo-eda-api, EDAAPI-Projections, EDAAPI-Projections-Vie

EDA-Projections, EDAProjectionLogEvent, pharo-eda-api, EDAAPI-Projections, EDAAPI-Proj
 EDA-Projections, EDAEventNotProjected, pharo-eda-api, EDAAPI-Projections, EDAAPI-Proje
 EDA-Projections, EDAEventProjected, pharo-eda-api, EDAAPI-Projections, EDAAPI-Projecti
 EDA-Projections, EDAProjectionRemoved, pharo-eda-api, EDAAPI-Projections, EDAAPI-Proje
 EDA-Projections, EDAProjectionSynchronizationChecked, pharo-eda-api, EDAAPI-Projection
 EDA-Projections, EDAProjectionRepository, pharo-eda-api, EDAAPI-Projections, EDAAPI-Pr
 EDA-Projections, EDAProjector, pharo-eda-api, EDAAPI-Projections, EDAAPI-Projections-P
 EDA-Projections, EDACompositeProjector, pharo-eda-api, EDAAPI-Projections, EDAAPI-Proj
 EDA-Projections, EDAMultipleProjectionsPerProjector, pharo-eda-api, EDAAPI-Projection
 EDA-Projections, EDASingleProjectionPerEventProjector, pharo-eda-api, EDAAPI-Projecti
 EDA-Projections-Voyage-Repository, EDAProjectionVoyageRepository, pharo-eda-adapters
 EDA-Projections-Voyage-Repository, EDAProjectionMemoryRepository, pharo-eda-adapters
 EDA-Projections-Voyage-Repository, EDAProjectionMongoRepository, pharo-eda-adapters,
 EDA-Projections-Voyage-Repository, EDAVOProjectionMongoRepository, pharo-eda-adapter
 EDA-SMTP, EDASmtpClient, pharo-eda-adapters, EDAA-SMTP, EDAA-SMTP, EDASmtpClient
 EDA-Settings, EDADUEnvironment, pharo-eda, EDA-Application, EDA-Application, EDADUEnvir
 EDA-Settings, EDASettings, pharo-eda-settings, EDA-Settings, EDA-Settings, EDASSettings
 EDA-StompMessaging, EDASTompTopicListener, pharo-eda-adapters, EDAA-Messaging-Stomp, E
 EDA-StompMessaging, EDACommandStompTopicListener, pharo-eda-adapters, EDAA-Messaging-
 EDA-StompMessaging, EDASTompTopicPublisher, pharo-eda-adapters, EDAA-Messaging-Stomp,
 EDA-StompMessaging, EDAEventStompTopicPublisher, pharo-eda-adapters, EDAA-Messaging-S
 EDA-Tools, AbstractEDAGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATAbstractG
 EDA-Tools, EDAAggregateContractCheckerGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Too
 EDA-Tools, EDACommandContractCheckerGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools
 EDA-Tools, EDAEventContractCheckerGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, E
 EDA-Tools, EDAGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATGenerator
 EDA-Tools, EDAAggregateGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATAggregat
 EDA-Tools, EDAApplicationGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATAppl
 EDA-Tools, EDACommandGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATCommandGen
 EDA-Tools, EDACompositeGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATComposit
 EDA-Tools, EDADefaultCommandBuilderGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools,
 EDA-Tools, EDADomainTestEventGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATDo
 EDA-Tools, EDADomainTestGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATDomainT
 EDA-Tools, EDAErrorsGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATErrorsGener
 EDA-Tools, EDAEventGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATEventGenerat
 EDA-Tools, EDAIntegrationTestGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATIn
 EDA-Tools, EDAREpositoryGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATReposit
 EDA-Tools, EDATestSupportGenerator, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATTestSu
 EDA-Tools, DefaultEnglishCommandNameResolver, pharo-eda-tools, EDAT-Tools, EDAT-Tools,
 EDA-Tools, DefaultEnglishEventNameResolver, pharo-eda-tools, EDAT-Tools, EDAT-Tools, ED
 EDA-Tools, EDACommandBrowser, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATCommandBrows
 EDA-Tools, EDACFuelHelper, pharo-eda-common, EDAC-Fuel, EDAC-Fuel, EDACFuelHelper
 EDA-Tools, EDATCheckerGenerator, pharo-eda-tools, EDAT-Tools, EDATCheckerGenerator
 EDA-Tools, EDATCheckerHelper, pharo-eda-tools, EDAT-Tools, EDAT-Tools, EDATCheckerHelpe
 EDA-Tools-Tests, AbstractEDAGeneratorTest, pharo-eda-tools, EDAT-Tools-Tests, EDAT-Too
 EDA-Traits, EDALoggingSettings, pharo-eda-common, EDAC-Logging, EDAC-Logging, EDACLoggi
 EDA-Traits, EDALoggingStub, pharo-eda-common, EDAC-Logging, EDAC-Logging, EDACLoggingSt
 EDA-Traits, EDAPrintOnHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACPrintOnHe
 EDA-Traits, EDASourceCodeHelperStub, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACSo
 EDA-Traits, EDATCollectionHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACColle

```

EDA-Traits, EDATDeepInspectHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACDeepIns
EDA-Traits, EDATErrorHandler, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACErrorHandler
EDA-Traits, EDATLanguageHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACLanguageHe
EDA-Traits, EDATLogging, pharo-eda-common, EDAC-Logging, EDAC-Logging, EDACLogging
EDA-Traits, EDATMongoHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACMongoHelper
EDA-Traits, EDATReflectionHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACReflecti
EDA-Traits, EDATRegexHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACRegexHelper
EDA-Traits, EDATRepository, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACRepository
EDA-Traits, EDATSettingsHelper, pharo-eda-common, EDAC-Settings, EDAC-Settings, EDACSettin
EDA-Traits, EDATSourceCodeHelper, pharo-eda-common, EDAC-Traits, EDAC-Traits, EDACSourceCo
EDA-Traits-Tests, EDATStringTest, pharo-eda-common, EDAC-Traits-Tests, EDAC-Traits-Tests, E
EDA-Traits-Tests, EDATCollectionHelperTest, pharo-eda-common, EDAC-Traits-Tests, EDAC-Tra
EDA-Traits-Tests, EDATLanguageHelperTest, pharo-eda-common, EDAC-Traits-Tests, EDAC-Trait
EDA-Traits-Tests, EDATReflectionHelperTest, pharo-eda-common, EDAC-Traits-Tests, EDAC-Tra
EDA-Traits-Tests, EDATSettingHelperTest, pharo-eda-common, EDAC-Settings-Tests, EDAC-Sett
EDA-Traits-Tests, EDATSourceCodeHelperTest, pharo-eda-common, EDAC-Traits-Tests, EDAC-Tra

```

13.1 Missing classes

- EDAEntity in EDA-Entities.
- EDATEqualsHelper in EDA-Traits.
- EDATHashHelper in EDA-Traits.

13.2 Classes not yet migrated

- EDATTestableProjection in EDA-Projections-Tests.
- EDATestEntityCounterProjection in EDA-Projections-Tests.

13.3 Extensions

- Pragma in EDA-Application.
- Dictionary in EDA-Commands.
- SmallInteger in EDA-Commands-Tests.
- Class in EDA-EventStore.
- ZnClient in EDA-Logging.
- ZnLogEvent in EDA-Logging.
- BSON in EDA-Messaging.
- Error in EDA-Messaging.

- `OID` in `EDA-Messaging`.
- `Collection` in `EDA-Projections`.
- `EDAProjection` in `EDA-Projections-Voyage-Repository`.
- `Dictionary` in `EDA-Serializers`.
- `EDAAggregateRoot` in `EDA-Serializers`.
- `EDACommand` in `EDA-Serializers`.
- `EDAEntity` in `EDA-Serializers`.
- `EDAEventSourcingEvent` in `EDA-Serializers`.
- `StampConstants` in `EDA-StompMessaging`.
- `StampMedium` in `EDA-StompMessaging`.
- `TStampFrameWithBody` in `EDA-StompMessaging`.
- `Dictionary` in `EDA-Tools`.

Bibliography

